

Exercices — Introduction à Node.js

Série d'exercices progressifs pour découvrir Node.js **par la pratique**.

Les deux premiers exercices sont en JavaScript pur — pour apprivoiser l'environnement.

À partir de l'exercice 3, tout est en TypeScript.

Les concepts JS/TS des modules précédents (async/await, Promises, types avancés) sont considérés acquis.

 **Date limite : 20 juin 2026**

Ressources utiles

Sujet	Lien
Documentation officielle Node.js	https://nodejs.org/fr/docs
Node.js — module <code>fs</code>	https://nodejs.org/api/fs.html
Node.js — module <code>path</code>	https://nodejs.org/api/path.html
Node.js — module <code>http</code>	https://nodejs.org/api/http.html
Node.js — module <code>stream</code>	https://nodejs.org/api/stream.html
Node.js — module <code>readline</code>	https://nodejs.org/api/readline.html
Node.js — module <code>process</code>	https://nodejs.org/api/process.html
Node.js — module <code>events</code>	https://nodejs.org/api/events.html
npm — documentation	https://docs.npmjs.com
<code>@types/node</code>	https://www.npmjs.com/package/@types/node

Installation & vérification

```
bash
```

```
node --version    # doit afficher v18+ recommandé
npm --version

# Pour les exercices TypeScript (à partir de l'exercice 3)
npm install -g ts-node typescript
npm install --save-dev @types/node
```

⚠ Note sur le passage JS → TS

Les exercices 1 et 2 se font dans des fichiers `.js`, exécutés avec `node fichier.js`.
À partir de l'exercice 3, tous les fichiers sont en `.ts`, exécutés avec `ts-node fichier.ts`.
Ton `tsconfig.json` doit avoir `"strict": true` et les options suivantes :

```
json
{
  "compilerOptions": {
    "strict": true,
    "target": "ES2022",
    "module": "CommonJS",
    "moduleResolution": "node",
    "esModuleInterop": true,
    "outDir": "./dist"
  }
}
```

Exercice 1 — L'environnement Node.js (JavaScript)

Difficulté: ★☆☆☆☆

Objectif

Comprendre ce qui change quand JavaScript tourne dans Node.js plutôt que dans un navigateur — et explorer les objets globaux disponibles.

Consignes

Étape 1 : Crée un fichier `exo1.js`. Sans installer quoi que ce soit, affiche dans la console :

- La version de Node.js en cours d'exécution
- Le répertoire courant d'exécution
- Le chemin absolu du fichier en cours d'exécution
- Les arguments passés en ligne de commande

Lance ensuite : `node exo1.js bonjour monde 42` et observe le résultat.

Étape 2 : Affiche les variables d'environnement disponibles. Puis lance le script en injectant une variable : `NOM=Alice node exo1.js` et affiche uniquement la variable `NOM` dans la console.

Étape 3 : Explore `process` : affiche la plateforme (`process.platform`), le PID du processus, et la quantité de mémoire utilisée (`process.memoryUsage()`).

Question : quels objets du navigateur sont absents dans Node.js ? (`window`, `document`, `fetch`...) Et inversement, que propose Node.js que le navigateur n'a pas ?

💡 **Hint :** dans Node.js, les variables globales utiles sont `process`, `__dirname`, `__filename`, et `global`. `process.argv` est un tableau — les deux premiers éléments sont toujours le chemin de `node` et celui du script. `process.env` est un objet contenant toutes les variables d'environnement.

Exercice 2 — Le système de modules (JavaScript)

Difficulté : ★★☆☆

Objectif

Comprendre le système de modules CommonJS (`require` / `module.exports`) et le comparer au système ES Modules (`import` / `export`) que tu connais déjà.

Consignes

Étape 1 : Crée un module `mathematiques.js` qui exporte :

- Une fonction `additionner(a, b)`
- Une fonction `multiplier(a, b)`
- Une constante `PI`

Importe-le dans `exo2.js` avec `require` et utilise chaque export.

Étape 2 : Crée un module `utilisateur.js` qui exporte **une classe** `Utilisateur` avec un constructeur `(nom, email)` et une méthode `présenter()`. Importe et instancie-la dans `exo2.js`.

Étape 3 : Observe ce qui se passe quand tu `require` le même module depuis deux fichiers différents — le module est-il réexécuté à chaque fois ? Cherche ce qu'est le **module cache** dans Node.js.

Étape 4 : Transforme `mathematiques.js` en ES Module (`.mjs` ou `"type": "module"` dans `package.json`). Qu'est-ce qui change dans la syntaxe ? Qu'est-ce que tu ne peux plus utiliser (`__dirname`, `__filename`...) ?

Exercice 3 — Le système de fichiers (TypeScript)

Difficulté: ★★☆☆☆

Objectif

Lire, écrire et manipuler des fichiers avec le module `fs` — en version synchrone, puis asynchrone avec Promises.

Consignes

Étape 1 : Crée un fichier `exo3.ts`. Crée un répertoire `données/` et écris-y un fichier `journal.txt` contenant trois lignes de texte. Lis ensuite ce fichier et affiche son contenu dans la console. Fais-le d'abord avec `fs.writeFileSync` / `fs.readFileSync`, puis refais-le avec `fs.promises.writeFile` / `fs.promises.readFile` et `async/await`.

Étape 2 : Écris une fonction `async listerFichiers(dossier: string): Promise<string[]>` qui retourne la liste des fichiers d'un répertoire (pas les sous-dossiers). Affiche le nom et la taille de chaque fichier en octets.

Étape 3 : Écris une fonction `async copierFichier(source: string, destination: string): Promise<void>` sans utiliser `fs.copyFile` — lis le contenu source et écris-le à destination. Gère les cas d'erreur : fichier source inexistant, répertoire destination inexistant.

Étape 4 : Écris une fonction `async rechercherDansFichier(chemin: string, motif: string): Promise<{ ligne: number; contenu: string }[]>` qui retourne toutes les lignes d'un fichier contenant le motif recherché, avec leur numéro de ligne.

💡 **Hint :** `fs/promises` expose toutes les fonctions asynchrones du module `fs`. Importe-le avec `import { promises as fs } from "fs"`. Pour typer les métadonnées d'un fichier, `fs.stat()` retourne un objet `Stats` — `stats.isFile()` et `stats.isDirectory()` permettent de distinguer fichiers et dossiers. Pour l'étape 4, `contenu.split("\n")` pour découper par ligne.

Exercice 4 — Le module `path` et la gestion des chemins (TypeScript)

Difficulté: ★★☆☆☆

Objectif

Manipuler des chemins de fichiers de façon portable (Windows / Linux / macOS) avec le module `path` — et comprendre pourquoi concaténer des chaînes est une mauvaise pratique.

Consignes

Étape 1 : Sans exécuter quoi que ce soit, prédis le résultat de chaque expression, puis vérifie :

```
ts

import path from "path"

path.join("/utilisateurs", "alice", "documents", "fichier.txt")
path.join("/utilisateurs/alice", "../bob", "fichier.txt")
path.resolve("relatif", "chemin")
path.basename("/utilisateurs/alice/fichier.txt")
path.basename("/utilisateurs/alice/fichier.txt", ".txt")
path.dirname("/utilisateurs/alice/fichier.txt")
path.extname("archive.tar.gz")
path.parse("/utilisateurs/alice/fichier.txt")
```

Étape 2 : Écris une fonction `async construireArborescence(racine: string, structure: object): Promise<void>` qui crée récursivement une arborescence de dossiers et fichiers à partir d'un objet JS :

```
ts

await construireArborescence("./projet", {
  "src": {
    "index.ts": "console.log('hello')",
    "utils": {
      "math.ts": "export const add = (a: number, b: number) => a + b"
    }
  },
  "package.json": JSON.stringify({ name: "projet", version: "1.0.0" }, null, 2),
  "README.md": "# Mon Projet"
})
```

Étape 3 : Écris une fonction `trouverFichiers(dossier: string, extension: string): Promise<string[]>` qui parcourt **récursivement** un dossier et retourne les chemins absolus de tous les fichiers ayant l'extension donnée.

💡 **Hint :** `path.join` construit un chemin en gérant les séparateurs selon l'OS.
`path.resolve` construit un chemin absolu depuis le répertoire courant. Pour la récursivité de l'étape 3, une fonction qui s'appelle elle-même avec `await` dans une boucle `for...of` est la solution la plus lisible. Cherche `fs.readdir` avec l'option `{`

`withFileTypes: true }` pour obtenir des objets `Dirent` qui exposent directement `isFile()` et `isDirectory()`.

Exercice 5 — Les Streams (TypeScript)

Difficulté: ★★☆☆

Objectif

Comprendre pourquoi les streams existent — et quand les utiliser plutôt que de lire un fichier entier en mémoire.

Consignes

Étape 1 : Génère un fichier `gros-fichier.txt` de 100 000 lignes (chaque ligne est `"Ligne N : [texte aléatoire]"`). Mesure la mémoire utilisée (`process.memoryUsage().heapUsed`) quand tu lis ce fichier avec `fs.readFile` (tout en mémoire) versus avec un `ReadStream`. Affiche la différence.

Étape 2 : Écris une fonction `async compterLignes(chemin: string): Promise<number>` qui compte les lignes d'un fichier **sans le charger entièrement en mémoire**, en utilisant le module `readline` et un `createReadStream`.

Étape 3 : Écris une fonction `async filtrerVersCSV(source: string, destination: string, prédicat: (ligne: string) => boolean): Promise<void>` qui lit un fichier ligne par ligne, garde uniquement les lignes qui passent le prédicat, et les écrit dans un fichier CSV destination — en streaming, sans tout charger en mémoire.

Étape 4 : Implémente un pipeline de transformation avec `stream.pipeline`: lis `gros-fichier.txt`, transforme chaque ligne en majuscules (avec un `Transform` stream), et écris le résultat dans `gros-fichier-majuscules.txt`. Compare les performances avec une approche `readFile` + `writeFile` classique.

💡 **Hint :** un stream est un flux de données traité par morceaux (*chunks*).

`fs.createReadStream` lit un fichier par blocs sans tout mettre en mémoire.

`readline.createInterface({ input: stream })` émet un événement `"line"` pour chaque ligne. Un `Transform` stream est un stream qui lit des données et en produit de nouvelles — il étend la classe `Transform` de `node:stream` et implémente `_transform(chunk, encoding, callback)`. `stream.pipeline(source, transform, destination, callback)` chaîne les streams et gère proprement la fermeture et les erreurs.

Exercice 6 — Les EventEmitter (TypeScript)

Difficulté: ★★☆☆

Comprendre le pattern EventEmitter natif de Node.js — la base de son architecture asynchrone — et le typer correctement en TypeScript.

Consignes

Étape 1 : Crée une classe `Minuteur` qui étend `EventEmitter` et émet :

- `"tick"` à chaque seconde avec le nombre de secondes écoulées
- `"terminé"` quand la durée est atteinte
- `"erreur"` si la durée passée est invalide (≤ 0)

ts

```
const minuteur = new Minuteur(5) // 5 secondes
minuteur.on("tick", (secondes) => console.log(`${secondes}s...`))
minuteur.on("terminé", () => console.log("Terminé !"))
minuteur.démarrer()
```

Étape 2 : Le typage par défaut de `EventEmitter` est permissif. Crée une interface `ÉvénementsMinuteur` et utilise le pattern de surcharge pour que `on("tick", handler)` type `handler` avec `(secondes: number) => void`, et `on("terminé", handler)` avec `() => void`.

Étape 3 : Crée une classe `FileDePriorité<T>` qui étend `EventEmitter` et gère une file d'attente avec priorités :

- `ajouter(élément: T, priorité: number): void` — émet `"ajout"`
- `traiterProchain(): T | undefined` — émet `"traitement"` ou `"vide"`
- `taille(): number`

Question : en quoi `EventEmitter` est-il différent du bus d'événements que tu as construit dans le module TypeScript avancé ? Quelles fonctionnalités manquent ? Lesquelles sont en plus ?

💡 **Hint :** pour typer un `EventEmitter`, la technique est de surcharger les signatures de `on`, `once`, et `emit` avec des overloads TypeScript. Cherche le pattern `interface TypedEventEmitter<Events extends Record<string, any[]>>`. Pour la `FileDePriorité`, un tableau trié par priorité décroissante suffit — utilise `Array.sort` après chaque `ajouter`.

Exercice 7 — Interface en ligne de commande (`TypeScript`)

Difficulté : ★★☆☆☆

Objectif

Construire un programme interactif en ligne de commande qui lit les entrées utilisateur et

gère des arguments typés — sans librairie externe.

Consignes

Étape 1 : Écris un parser d'arguments de ligne de commande. Il doit transformer `process.argv` en un objet structuré :

```
bash
```

```
ts-node cli.ts --port 3000 --env production --verbose fichier.txt
```

```
ts
```

```
// doit produire :
```

```
{
  flags: { port: "3000", env: "production", verbose: true },
  positionnels: ["fichier.txt"]
}
```

Typier le résultat avec un générique pour que les flags soient accessibles avec autocomplétion selon un schéma défini à l'avance.

Étape 2 : Écris un programme interactif avec `readline` qui :

- Affiche un prompt `>`
- Accepte des commandes : `aide`, `heure`, `quitter`, et `écho [message]`
- Répond à chaque commande
- Se termine proprement sur `quitter` ou `Ctrl+C`

Étape 3 : Ajoute une commande `lire [chemin]` qui lit un fichier et l'affiche dans la console, et `écrire [chemin] [contenu]` qui crée ou écrase un fichier. Gère les erreurs (fichier inexistant, permissions) avec des messages clairs pour l'utilisateur — pas de stack trace brut.

💡 Hint : `readline.createInterface({ input: process.stdin, output: process.stdout })` crée une interface interactive. La méthode `question(prompt, callback)` affiche un prompt et attend une réponse. Pour un prompt en boucle, rappelle la fonction d'écoute dans le callback. `process.on("SIGINT", () => { ... })` intercepte `Ctrl+C`. Pour le parser d'arguments, itère sur `process.argv.slice(2)` et détecte si un élément commence par `--`.

Exercice 8 — Serveur HTTP natif (TypeScript)

Difficulté : ★★☆☆☆

Objectif

Consignes

Étape 1 : Crée un serveur qui écoute sur le port `3000` et répond :

- `GET /` → `200 { message: "Bienvenue" }`
- `GET /santé` → `200 { statut: "ok", timestamp: Date.now() }`
- Toute autre route → `404 { erreur: "Route introuvable" }`

Toutes les réponses sont en JSON avec le header `Content-Type: application/json`.

Étape 2 : Ajoute le support des paramètres de route et de query string. Implémente un mini-router qui mappe des patterns vers des handlers :

```
ts

router.get("/utilisateurs/:id", (req, res, params) => {
  const { id } = params // "id" est extrait de l'URL
  res.json({ id, nom: "Alice" })
})

router.get("/recherche", (req, res, params, query) => {
  const { q } = query // ?q=alice
  res.json({ résultats: [], requête: q })
})
```

Étape 3 : Ajoute le support des requêtes `POST` avec un body JSON. Écris une fonction `lireBody(req: IncomingMessage): Promise<unknown>` qui lit le stream de la requête et parse le JSON. Ajoute une route `POST /utilisateurs` qui valide que le body contient `nom` et `email`, et retourne l'utilisateur créé avec un `id` auto-incrémenté.

Étape 4 : Ajoute un système de middlewares qui s'exécutent avant chaque handler :

- Un middleware de logging : `[GET] /utilisateurs/1 - 200 - 3ms`
- Un middleware CORS : ajoute les headers appropriés
- Un middleware d'authentification basique : vérifie le header `Authorization: Bearer token123` sur toutes les routes sauf `/santé`

 **Hint :** `http.createServer((req, res) => { })` crée un serveur. `req.url` contient le chemin + query string — le module `url` ou `new URL(req.url, "http://localhost")` permet de les séparer proprement. Pour lire le body d'un `POST`, écoute les événements `"data"` et `"end"` sur `req` : accumule les chunks dans un tableau et parse à la fin. Un middleware est une fonction `(req, res, next) => void` — `next()` passe au suivant.

Difficulté : ★ ★ ★ ★ ☆

Langage : TypeScript

Contexte

Tu vas construire un outil en ligne de commande complet — utilisable directement dans le terminal comme une vraie commande installée — pour gérer des notes persistées dans des fichiers JSON locaux.

Comportement attendu

```
bash

# Créer une note
notes ajouter "Réunion demain à 14h" --tag travail --priorité haute

# Lister les notes
notes lister
notes lister --tag travail
notes lister --priorité haute

# Rechercher
notes chercher "réunion"

# Marquer comme faite
notes terminer 3

# Supprimer
notes supprimer 3

# Exporter
notes exporter --format json > export.json
notes exporter --format csv > export.csv

# Statistiques
notes stats
```

Partie 1 — Le modèle de données

Définis les types :

```
ts
```

```

type Priorité = "basse" | "normale" | "haute"
type StatutNote = "active" | "terminée"

interface Note {
  id: number
  contenu: string
  tags: string[]
  priorité: Priorité
  statut: StatutNote
  crééeÀ: string // ISO 8601
  modifiéeÀ: string
}

```

Le fichier de stockage est `~/ .notes/notes.json`. S'il n'existe pas au premier lancement, le créer automatiquement.

Partie 2 — Le parser de commandes

Écris un parser qui extrait depuis `process.argv`:

- La commande (`ajouter`, `lister`, `chercher`, `terminer`, `supprimer`, `exporter`, `stats`)
- Les arguments positionnels (le contenu de la note)
- Les flags (`--tag`, `--priorité`, `--format`)

Si la commande est inconnue ou qu'un argument requis manque, afficher un message d'aide clair et quitter avec le code `process.exit(1)`.

Partie 3 — Le service de notes

Crée une classe `ServiceNotes` avec:

- `charger(): Promise<Note[]>` — lit le fichier JSON
- `sauvegarder(notes: Note[]): Promise<void>` — écrit le fichier JSON (formaté)
- `ajouter(contenu: string, tags: string[], priorité: Priorité): Promise<Note>`
- `lister(filtres?: { tag?: string; priorité?: Priorité; statut?: StatutNote }): Promise<Note[]>`
- `chercher(requête: string): Promise<Note[]>` — recherche insensible à la casse dans le contenu et les tags
- `terminer(id: number): Promise<Note>`
- `supprimer(id: number): Promise<void>`
- `exporter(format: "json" | "csv"): Promise<string>` — retourne la chaîne formatée
- `stats(): Promise<{ total: number; terminées: number; parPriorité: Record<Priorité, number> }>`

L'affichage dans le terminal doit être lisible :

- Les notes sont affichées avec leur id, contenu, tags, priorité et statut
- Un indicateur visuel de priorité : `[!]` haute, `[ ]` normale, `[.]` basse
- Les notes terminées sont affichées différemment (préfixe `[✓]`)
- Les erreurs sont affichées sur `process.stderr` (pas `stdout`)

Partie 5 — Installation locale

Configure `package.json` pour que la commande soit exécutable directement :

```
json
{
  "bin": { "notes": "./dist/index.js" }
}
```

Ajoute un `#!/usr/bin/env node` en première ligne du fichier compilé. La commande `npm link` doit rendre `notes` disponible globalement dans le terminal.

Critères de succès

- ☐ `notes ajouter "Ma note" --tag dev --priorité haute` crée une note persistée
 - ☐ Les données survivent à la fermeture du terminal (stockage JSON)
 - ☐ `notes lister --tag dev` filtre correctement
 - ☐ `notes exporter --format csv` produit un CSV valide
 - ☐ Les erreurs (id inexistant, fichier corrompu) affichent un message clair sans stack trace
 - ☐ Aucun `any` — `tsc --strict --noEmit` sans erreur
-

Mini-projet 2 — API REST sans framework

Difficulté: ★ ★ ★ ★ ★

Langage: TypeScript

Contexte

Tu vas construire une API REST complète **sans Express, sans Fastify, sans aucun framework HTTP** — uniquement avec le module natif `http` de Node.js et les concepts vus dans les exercices. Le domaine : une API de gestion de tâches (to-do list) avec utilisateurs et authentification basique.

La contrainte principale est architecturale : ton code doit ressembler à ce qu'un framework ferait sous le capot — avec un router, des middlewares, des controllers, et une couche de persistance.

Partie 1 — Le router

Construis un `Router` qui supporte :

- Les méthodes HTTP : `GET`, `POST`, `PUT`, `PATCH`, `DELETE`
- Les paramètres de route : `/tâches/:id`
- Les middlewares globaux et par route
- La gestion automatique des routes `404` et des erreurs `500`

```
ts

const router = new Router()

router.use(loggerMiddleware)
router.use(corsMiddleware)

router.get("/tâches", [authMiddleware], tâchesController.lister)
router.post("/tâches", [authMiddleware, validerBodyMiddleware(CréerTâcheDTO)], tâchesC
router.get("/tâches/:id", [authMiddleware], tâchesController.trouver)
router.patch("/tâches/:id", [authMiddleware], tâchesController.modifier)
router.delete("/tâches/:id", [authMiddleware], tâchesController.supprimer)
```

Partie 2 — Les types du contexte

Chaque handler reçoit un objet `Contexte` typé :

```
ts

interface Contexte {
  req: IncomingMessage
  res: ServerResponse
  params: Record<string, string> // paramètres de route
  query: Record<string, string> // query string
  body: unknown // body parsé
  utilisateur?: UtilisateurSession // injecté par authMiddleware
}

type Handler = (ctx: Contexte) => Promise<void>
type Middleware = (ctx: Contexte, suivant: () => Promise<void>) => Promise<void>
```

Ajoute des méthodes utilitaires sur `Contexte` :

- `ctx.json(données, status?)` — répond en JSON
- `ctx.erreur(message, status?)` — répond avec une erreur formatée

- `ctx.lireBody<T>(): Promise<T>` — lit et parse le body

Partie 3 — La persistance

Utilise des fichiers JSON comme base de données (comme dans le mini-projet 1). Crée un `Repository<T extends { id: number }>` générique qui expose :

- `trouverTous(filtre?: Partial<T>): Promise<T[]>`
- `trouverParId(id: number): Promise<T | undefined>`
- `créer(données: Omit<T, "id" | "crééÀ" | "modifiéeÀ">): Promise<T>`
- `modifier(id: number, données: Partial<T>): Promise<T>`
- `supprimer(id: number): Promise<boolean>`

Chaque entité est stockée dans son propre fichier : `utilisateurs.json`, `tâches.json`.

Partie 4 — Les endpoints

Implémente les routes suivantes :

Authentification

```
POST /auth/inscription { nom, email, motDePasse } → { token, utilisateur }
POST /auth/connexion  { email, motDePasse }      → { token, utilisateur }
```

Le token est un simple JWT-like encodé en Base64 (pas besoin de librairie JWT) :

```
base64(JSON.stringify({ userId, email, exp: Date.now() + 86400000 })))
```

Tâches (toutes protégées par `authMiddleware`)

```
GET    /tâches          → liste les tâches de l'utilisateur connecté
POST   /tâches          → créer une tâche
GET    /tâches/:id      → détail d'une tâche
PATCH /tâches/:id      → modifier partiellement
DELETE /tâches/:id      → supprimer
GET    /tâches/stats    → { total, terminées, enCours, parPriorité }
```

Partie 5 — Les middlewares

Implémente ces middlewares :

`loggerMiddleware` — logue chaque requête : `[2024-01-15 14:32:01] POST /tâches - 201 - 12ms`

`corsMiddleware` — ajoute les headers CORS appropriés

`authMiddleware` — vérifie le header `Authorization: Bearer <token>`, décode le token, vérifie qu'il n'est pas expiré, injecte `ctx.utilisateur`

`validerBodyMiddleware(Classe)` — instancie la classe avec le body reçu et vérifie que les champs requis sont présents et du bon type. Utilise le système de décorateurs de validation du module TypeScript avancé si tu l'as implémenté, sinon une validation manuelle.

`gestionErreurMiddleware` — middleware global qui attrape toutes les erreurs non gérées et retourne une réponse `500` formatée sans exposer le stack trace.

Partie 6 — Script de test

Écris un script `tester.ts` qui effectue une séquence complète via `fetch` (Node 18+) :

1. Inscrit un utilisateur
2. Se connecte et récupère le token
3. Crée 3 tâches
4. Liste les tâches
5. Modifie le statut d'une tâche
6. Supprime une tâche
7. Récupère les statistiques
8. Essaie d'accéder sans token → vérifie que c'est bien une `401`

Chaque étape affiche le résultat et un  ou  selon le succès.

Critères de succès

- ☐ Le router parse correctement les paramètres `/tâches/42` → `params.id === "42"`
- ☐ `authMiddleware` rejette les requêtes sans token ou avec token expiré
- ☐ Les tâches d'un utilisateur ne sont pas visibles par un autre
- ☐ Le script `tester.ts` passe toutes ses vérifications
- ☐ Les erreurs 404, 401, 400, 500 sont toutes formatées en JSON (jamais du HTML)
- ☐ Aucun `any` — `tsc --strict --noEmit` sans erreur
- ☐ Le serveur gère proprement `SIGINT` (Ctrl+C) : `log "Arrêt du serveur"` et quitte